



**NATIONAL RESEARCH UNIVERSITY OF INFORMATION
TECHNOLOGY, MECHANICS AND OPTICS**

FACULTY OF CONTROL SYSTEMS AND ROBOTICS

LABORATORY TASK N°3

COMPLETED BY: Valverde Ramírez, Edgar David

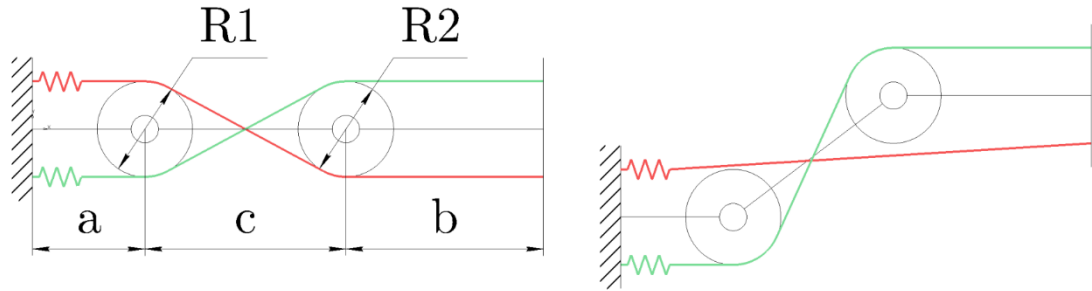
ISU: 520193

DISCIPLINE: Simulation of Robotic Systems

INSTRUCTOR: PhD. Ivan Borisov

1. Data:

The primary objective of this laboratory task is to model and simulate a passive mechanism using the MuJoCo physics engine. The mechanism consists of a cable-driven system with specific geometric constraints.



The specific system parameters provided for the simulation are listed in the table below:

Parameter	Value
R1	0.01
R2	0.026
a	0.037
b	0.052
c	0.045

2. Modeling:

To simulate the mechanism, the system is defined in XML format. Based on the design schematics, the mechanism comprises a rigid base on the left side, followed by a chain of three links connected by two hinge joints. Cylinders are placed at the articulations to function as pulleys, and an end-effector is defined as the anchor point for the tendons.

Rigid Body Structure

We begin by defining the rigid elements and their hierarchy. To prevent simulation conflicts, collision detection is disabled (`contype="0"`, `conaffinity="0"`) for the links. This ensures that the links do not intersect physically with the pulleys (cylinders) or the tendons, as they are superimposed in the same plane.

The hierarchy is defined as follows:

```

base
├── link1 (contype="0")
│   └── link2 (contype="0")
│       ├── joint1 (hinge)
│       ├── cyl1
│       └── link3 (contype="0")
│           ├── joint2 (hinge)
│           ├── cyl2
│           └── end_effector

```

Tendon Routing and Constraints

The tendons are routed in a "crisscross" pattern, connecting the base to the end-effector parallel to the pulleys. Specific site elements are defined to act as anchor and wrapping points:

Base (anchor points)

```

├── anchor1a
└── anchor1b

```

end_effector (anchor points)

```

├── anchor2a
└── anchor2b

```

link2 (cyl1, wrapping sites)

```

├── wrap1a
├── wrap1b
└── center1 (joint1 center)

```

Link3 (cyl2, wrapping sites)

```

├── wrap2a
├── wrap2b
└── center2 (joint2 center)

```

In MuJoCo, a spatial tendon path typically follows the structure: Site -> Geom -> Site. However, this mechanism requires the tendon to wrap around *two* consecutive pulleys (geoms) without an intermediate fixed site, which violates standard definition rules.

```

Site: anchor1a
↓
Geom: cyl1 / wrap1a
↓
Geom: cyl2 / wrap2b
↓
Site: anchor2b

```

To resolve this, two auxiliary bodies (mid1, mid2) were introduced. These bodies contain anchor sites (mid1_site, mid2_site) and are constrained using slide joints in the X and Z axes.

```
mid1 (punto de anclaje)
├── mid1_site
├── mid1_x (joint: slide x)
└── mid1_z (joint: slide z)
```

```
mid2 (punto de anclaje)
├── mid2_site
├── mid2_x (joint: slide x)
└── mid2_z (joint: slide z)
```

To ensure these auxiliary points effectively facilitate the tendon path without altering the kinematics, equality constraints were applied. These constraints connect the auxiliary sites to the centers of the joints (center1, center2), effectively "pinning" the tendon segment between the two pulleys to the articulation centers while satisfying the Site -> Geom -> Site requirement.

```
<equality>
  <connect site1="mid1_site" site2="center1"/>
  <connect site1="mid1_site" site2="center2"/>
  <connect site1="mid2_site" site2="center1"/>
  <connect site1="mid2_site" site2="center2"/>
</equality>
```

Final Tendon Structure:

```
<tendon>
  <spatial name="spring1" width="0.001" rgba="1 0 0 1"
stiffness="10" damping="10" springlength="0.005">
    <site site="anchor1a"/>
    <geom geom="cyl1" sidesite="wrap1a"/>
    <site site="mid1_site"/>
    <geom geom="cyl2" sidesite="wrap2b"/>
    <site site="anchor2b"/>
  </spatial>

  <spatial name="spring2" width="0.001" rgba="0 1 0 1"
stiffness="10" damping="10" springlength="0.005">
    <site site="anchor1b"/>
    <geom geom="cyl1" sidesite="wrap1b"/>
```

```

        <site site="mid2_site"/>
        <geom geom="cyl2" sidesite="wrap2a"/>
        <site site="anchor2a"/>
    </spatial>
</tendon>

```

Finally, actuators were added to joint1 and joint2 to allow for active control testing during the simulation:

```

<actuator>
    <motor joint="joint1" ctrlrange="-1 1" ctrllimited="true"/>
    <motor joint="joint2" ctrlrange="-1 1" ctrllimited="true"/>
</actuator>

```

3. Python Simulation

The simulation is executed using the MuJoCo Python bindings. The process involves loading the model, initializing the data structures, and running a time-stepped loop.

The tendon.xml model is loaded, and the ID for the end-effector site is retrieved for tracking.

```

model = mujoco.MjModel.from_xml_path("tendon.xml")
data = mujoco.MjData(model)
site_id = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_SITE,
SITE_NAME)

```

The simulation runs for NSTEPS. At each step, a sinusoidal torque is applied to the joints to induce movement. The X and Z positions of the end-effector are recorded at every time step for analysis.

```

with mujoco.viewer.launch_passive(model, data) as viewer:
    viewer.cam.azimuth = 90
    viewer.cam.elevation = -30
    viewer.cam.distance = 0.4
    viewer.cam.lookat = np.array([0.05, 0.0, 0.1])

    time.sleep(2)

    for i in range(NSTEPS):
        if viewer.is_running():
            t = data.time
            data.ctrl[0] = 0.01*np.sin(50 * t)

```

```

data.ctrl[1] = -0.01*np.cos(50 * t)

pos = data.site_xpos[site_id].copy()
positions_x.append(pos[0])
positions_z.append(pos[2])
times.append(t)

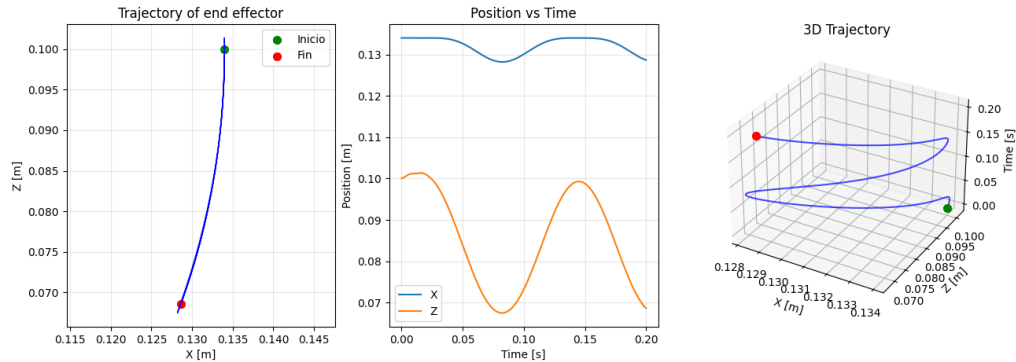
mujoco.mj_step(model, data)
viewer.sync()
time.sleep(0.001)

viewer.close()

```

4. Resultados:

The simulation generated the following trajectory data for the end-effector.



As observed in the generated graphs, the end-effector successfully traces a smooth curve, validating the tendon constraints and the auxiliary point implementation.

5. Conclusions

The following conclusions are drawn from the modeling and simulation of the passive tendon-driven mechanism:

- **Successful Geometric Modeling:** The mechanism was correctly implemented in MuJoCo using the specific parameters provided (R1, R2, a, b, c). The hierarchical structure of the rigid bodies (links) and joints accurately represents the desired kinematic chain.
- **Resolution of Tendon Constraints:** A key challenge in this task was MuJoCo's limitation regarding spatial tendons, which requires a site-geom-site structure and does not natively support wrapping around two consecutive pulleys. This was successfully resolved by introducing auxiliary "mid-bodies" constrained by slide joints and equality constraints. This method effectively served as a virtual anchor, allowing the tendon to route correctly between the two pulleys without violating the simulation engine's rules.

- **Validation of Kinematics:** The resulting simulation demonstrates that the tendon configuration works as intended. The movement of the end-effector is smooth and consistent with the input torques applied to the joints, confirming that the auxiliary constraints provided a stable and physically plausible solution for the cable transmission.

6. XML

```
<?xml version="1.0" ?>
<mujoco>
  <option timestep="1e-4" integrator="RK4" gravity="0 0 0"/>

  <default>
    <joint damping="0.001"/>
  </default>

  <worldbody>
    <light pos="0 0 3" dir="0 0 -1"/>
    <geom type="plane" size="1 1 0.1" rgba=".9 0 0 1"/>

    <body name="mid1" pos="0.063 0 0.1">
      <site name="mid1_site" pos="0 0 0" type="sphere" size="0.001"/>
      <joint name="mid1_x" type="slide" axis="1 0 0"/>
      <joint name="mid1_z" type="slide" axis="0 0 1"/>
      <geom type="sphere" size="0.002" rgba="1 0 0 1"/>
    </body>

    <body name="mid2" pos="0.063 0 0.1">
      <site name="mid2_site" pos="0 0 0" type="sphere" size="0.001"/>
      <joint name="mid2_x" type="slide" axis="1 0 0"/>
      <joint name="mid2_z" type="slide" axis="0 0 1"/>
      <geom type="sphere" size="0.002" rgba="0 1 0 1"/>
    </body>

    <body name="base" pos="0 0 0.1">
      <geom type="cylinder" size=".0005 .005" rgba=".8 .3 .3 1"/>
      <site name="anchor1a" pos="0 0 0.005" type="sphere"
size="0.002"/>
      <site name="anchor1b" pos="0 0 -0.005" type="sphere"
size="0.002"/>
      <body name="link1" pos="0 0 0">
        <geom name="link1_geom" type="cylinder" size=".0005 .0185"
fromto="0 0 0 0.037 0 0" rgba=".8 .3 .3 1" contype="0" conaffinity="0"/>
        <body name="link2" pos="0.037 0 0">
```

```

0"/>
    <joint name="joint1" type="hinge" axis="0 1 0" pos="0 0
0"/>
    <geom name="link2_geom" type="cylinder" size=".0005
0.026" fromto="0 0 0 0.052 0 0" rgba=".3 .8 .3 1" contype="0"
conaffinity="0"/>
    <geom name="cyl1" type="cylinder" size=".005 .005"
euler="90 0 0" rgba=".3 .3 .3 0.5"/>
    <site name="wrap1a" pos="0 0 0.01" type="sphere"
size="0.001"/>
    <site name="wrap1b" pos="0 0 -0.01" type="sphere"
size="0.001"/>
    <site name="center1" pos="0 0 0" type="sphere"
size="0.001"/>
    <body name="link3" pos="0.052 0 0">
    <joint name="joint2" type="hinge" axis="0 1 0"
pos="0 0 0"/>
    <geom name="link3_geom" type="cylinder" size=".0005
.005" fromto="0 0 0 0.045 0 0" rgba=".3 .3 .8 1" contype="0"
conaffinity="0"/>
    <geom name="cyl2" type="cylinder" size=".013 .005"
euler="90 0 0" rgba=".3 .3 .3 0.5"/>
    <site name="wrap2a" pos="0 0 0.017" type="sphere"
size="0.001"/>
    <site name="wrap2b" pos="0 0 -0.017" type="sphere"
size="0.001"/>
    <site name="center2" pos="0 0 0" type="sphere"
size="0.001"/>
    <body name="end_effector" pos="0.045 0 0">
    <geom type="cylinder" size=".0005 .013" rgba=".3
.3 .8 1"/>
    <site name="anchor2a" pos="0 0 .013"
type="sphere" size="0.002"/>
    <site name="anchor2b" pos="0 0 -.013"
type="sphere" size="0.002"/>
    <site name="end_effector" pos="0 0 0"
type="sphere" size="0.001"/>
    </body>
  </body>
</body>
</worldbody>

<tendon>

```



```

    <spatial name="spring1" width="0.001" rgba="1 0 0 1" stiffness="10"
damping="10" springlength="0.005">
        <site site="anchor1a"/>
        <geom geom="cyl1" sidesite="wrap1a"/>
        <site site="mid1_site"/>
        <geom geom="cyl2" sidesite="wrap2b"/>
        <site site="anchor2b"/>
    </spatial>

    <spatial name="spring2" width="0.001" rgba="0 1 0 1" stiffness="10"
damping="10" springlength="0.005">
        <site site="anchor1b"/>
        <geom geom="cyl1" sidesite="wrap1b"/>
        <site site="mid2_site"/>
        <geom geom="cyl2" sidesite="wrap2a"/>
        <site site="anchor2a"/>
    </spatial>
</tendon>

<equality>
    <connect site1="mid1_site" site2="center1"/>
    <connect site1="mid1_site" site2="center2"/>
    <connect site1="mid2_site" site2="center1"/>
    <connect site1="mid2_site" site2="center2"/>
</equality>

<actuator>
    <motor joint="joint1" ctrlrage="-1 1" ctrllimited="true"/>
    <motor joint="joint2" ctrlrage="-1 1" ctrllimited="true"/>
</actuator>
</mujoco>

```

7. Python code

```

import mujoco
import mujoco.viewer
import matplotlib.pyplot as plt
import numpy as np
import time
import glfw

NSTEPS = 2000
SITE_NAME = "end_effector"

```

```

model = mujoco.MjModel.from_xml_path("tendon.xml")
data = mujoco.MjData(model)
site_id = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_SITE, SITE_NAME)

print(f" - DOFs: {model.nq}")
print(f" - Number of bodys: {model.nbody}")
print(f" - Number of joints: {model.njnt}")
print(f" - Number of actuators: {model.nu}")

positions_x = []
positions_z = []
times = []

with mujoco.viewer.launch_passive(model, data) as viewer:
    viewer.cam.azimuth = 90
    viewer.cam.elevation = -30
    viewer.cam.distance = 0.4
    viewer.cam.lookat = np.array([0.05, 0.0, 0.1])

    time.sleep(2)

    for i in range(NSTEPS):
        if viewer.is_running():
            t = data.time
            data.ctrl[0] = 0.01*np.sin(50 * t)
            data.ctrl[1] = -0.01*np.cos(50 * t)

            pos = data.site_xpos[site_id].copy()
            positions_x.append(pos[0])
            positions_z.append(pos[2])
            times.append(t)

            mujoco.mj_step(model, data)
            viewer.sync()
            time.sleep(0.001)

    viewer.close()

plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.plot(positions_x, positions_z, 'b-', linewidth=1)
plt.scatter(positions_x[0], positions_z[0], color='green', s=50,
label='Start')

```

```
plt.scatter(positions_x[-1], positions_z[-1], color='red', s=50,
label='End')
plt.xlabel('X [m]')
plt.ylabel('Z [m]')
plt.title(f'Trajectory of end effector')
plt.legend()
plt.grid(True, alpha=0.3)
plt.axis('equal')

# Subplot 2: Position vs Time
plt.subplot(1, 3, 2)
plt.plot(times, positions_x, label='X')
plt.plot(times, positions_z, label='Z')
plt.xlabel('Time [s]')
plt.ylabel('Position [m]')
plt.title('Position vs Time')
plt.legend()
plt.grid(True, alpha=0.3)

# Subplot 3: 3D Trajectory
ax = plt.subplot(1, 3, 3, projection='3d')
ax.plot(positions_x, positions_z, times, 'b-', alpha=0.7)
ax.scatter(positions_x[0], positions_z[0], times[0], color='green', s=50)
ax.scatter(positions_x[-1], positions_z[-1], times[-1], color='red', s=50)
ax.set_xlabel('X [m]')
ax.set_ylabel('Z [m]')
ax.set_zlabel('Time [s]')
ax.set_title('3D Trajectory')

plt.show()
```